Embedded Operating System and Architecture

Stridi Simone

Anno Accademico 2026/2027

Contents

1 Boot and CPU Configuration **5**

1.1 Introduction to Low-Level CPU Control 5

1.1.1 Where You Start and Where You End 5

1.2 Memory Technologies in Embedded Systems 6

1.2.1 SRAM (Static RAM): The Working Memory 6

1.2.2 NOR Flash: Execute-in-Place Memory 7

1.2.3 Contrasting with NAND Flash and Block Devices 7

1.2.4 Applying Constraints to C Code 8

1.2.5 The Linker Script and the Memory Map 8

MEMORY: Defining the Regions 9

1.2.6 SECTIONS: Building the Executable Image 9

Run Address and Load Address: The `.data` Section 10

bss, Heap, and Stack: RAM With No Stored Image 10

1.2.7 Linker-Defined Symbols Are Addresses, Not Variables 11

1.2.8 Reading the Result: The Map File, `nm`, and `objdump` 11

Analyzing the Map File (`.map`) 12

Analyzing the Final Executable (`.axf`) 13

Analyzing the Object Files (`.o`) 14

Pointers, Local, and Static Variables 15

Try It: Testing Global Declarations 15

Pointers in RAM and Little-Endianness 16

Local vs. Static Local Variables 16

1.2.9 Advanced Memory Layout: Custom Sections and Orphans . . 16

1.2.10 Overflowing a Region 17

1.3 Reset: The Hardware Contract and `boot.s` 18

1.3.1 The Assembly Environment 18

1.3.2 The Vector Table 18

The Thumb Bit Constraint 19

KEEP and Garbage Collection 19

1.3.3 The Reset Handler 19

1.3.4 Back to the Base: Why Only One Line Prints 20

1.4 Taking Control of the CPU 21

1.4.1 What is a Debugger? 21

1.4.2 A Remote Debugger: Two Programs, Two Terminals 22

Reconstructing the Debug Target 22

The GDB Commands You Need 22

1.4.3 Attaching at Reset 22

1.4.4 Cortex-M3 Programmer's Model: The Registers 23

The Status Register (xPSR) and CPU Modes 23
1.4.5 ARM Assembly Basics: A Load/Store Architecture 24
Constants, Labels, and Jumps 24
Control Flow: Decisions, Bits, and Interrupts 25
Assembler Directives 25
1.5 Writing the Startup Script (`Reset_Handler`) 26
1.5.1 Steps 1 and 2: Mask Interrupts and Set the Stack 26
1.5.2 Step 3: Copy `.data` from FLASH to RAM 27
Watching the Copy Loop in GDB 28
Try It: Breaking the Stack on Purpose 28
Test 1: Address Outside of RAM (`0x30000000`) 28
Test 2: Unaligned Stack Pointer (`0x20000001`) 29
1.5.3 Step 4: Zero the `.bss` Section 29
1.6 Exceptions and Faults: One Family 30
1.6.1 Completing the Vector Table with Weak Symbols 30
The Effect on Memory Layout 31
1.6.2 CPU Settings Live at Memory Addresses 31
The Read-Modify-Write Pattern and `volatile` 32
1.6.3 Step 5: VTOR, Where the Vector Table Is 32
1.6.4 Step 6: SHCSR, Separate Fault Handlers 33
1.6.5 Step 7: CCR, What Counts as a Fault 33
1.6.6 Step 8: Exception Priorities 34
1.6.7 Step 9 and After: PRIMASK and Handing Over to C 35
1.6.8 Checking the Configuration from GDB 35
1.6.9 Catching a Divide by Zero 36
Triggering the Fault 37
Validating the Configuration Steps 37
1.6.10 What Real Chips Add (`SystemInit`) 37
1.7 Life Without an Operating System 38
1.7.1 Hosted vs. Freestanding C Environments 38
What is Available in Freestanding C? 38
1.7.2 What is Really Under `printf` 39
1.7.3 Why This Course Uses No libc At All 40
1.7.4 Bypassing `libc`: The `printf` Problem 40
1.7.5 Semihosting: How the Program Still Prints 40
1.7.6 Implementing Semihosting in C (`main.c`) 41
1.7.7 Semihosting is a Trap 41
1.7.8 Privileged and Unprivileged Code on Cortex-M 42
The CONTROL Register 42
1.7.9 Dropping Privilege: A One-Way Street 42
1.7.10 What Unprivileged Code Cannot Do (The Sandbox Constraints) 43
Try It: The HardFault Escalation 43
Debugging the CONTROL Register 44
1.7.11 SVC: The Door That Stays Open 44
1.7.12 This Is a System Call 44
1.7.13 Adding the System Call 45
1.7.14 Touching the Hardware Without Privilege 45
Conclusion: Privilege and CPU Modes 46

1.8 Conclusion: The Final Result 46
1.9 Appendix: Where to Find the Code 47
1.9.1 The Base Skeleton 47
1.9.2 The Finished Result 47

Chapter 1

Boot and CPU Configuration

1.1 Introduction to Low-Level CPU Control

In the previous week, we built a tiny program for a Cortex-M3, ran it in QEMU, and saw it print one line[cite: 101]. Several pieces of that program were handed to us as-is: the linker script that decided where everything goes in memory, and the startup code that ran before `main()`. We were asked to trust them. This phase is about stopping that trust and building those components from scratch. By the end of this phase, developers will understand, and will have written, every instruction that the CPU executes between power-on and the first line of `main()`[cite: 101]. The development process covers knowing where each piece of a C program lives in the chip's memory and why, how the CPU finds its first instruction, how to watch the CPU one instruction at a time with a debugger, how to configure the CPU so that errors are reported clearly instead of silently, and what it means to run C with nothing underneath it (no C library, no operating system)[cite: 101]. Finally, the program will be split into a privileged and an unprivileged part, building the smallest possible version of a system call (`svc`)[cite: 101].

The process is organized into three core units[cite: 101]:

1. **Where code and data live:** The kinds of memory a microcontroller has, the Cortex-M memory map, how the linker script turns that map into decisions, and what the CPU does at reset[cite: 101].

2. **Taking control of the CPU:** The debugger, the CPU registers, just enough ARM assembly to read and write boot code, and writing the reset handler step by step (stack, `.data`, `.bss`, vector table, fault configuration, priorities)[cite: 101].

3. **Life without an OS underneath:** What "freestanding" C means, how the program prints without a C library, privileged and unprivileged code, and the `svc` instruction as a system call[cite: 101].

1.1.1 Where You Start and Where You End

All the in-class work happens on top of a starting skeleton, specifically the folder `week-02/base/`[cite: 101]. It is derived from the previous week with one major difference: the original C startup code, which did all the setup work, has been thrown away[cite: 101].

In its place is a new, almost empty assembly file (`base/boot/startup.s`)[cite: 101]. It contains only the two words the CPU absolutely needs at reset (a stack address and the address of the first instruction), and a `Reset_Handler` that does nothing except call `main()` and then loop forever if `main()` ever returns[cite: 101].

The program in `base/src/main.c` attempts to print two lines[cite: 101]:

1. A standard string literal[cite: 101].
2. `data_message`, a global character array initialized with the text `"main: this text lives in .data\n"`[cite: 101].

If we build and run the base skeleton (`make` and `make run`), we see only one line[cite: 101]:

```
main: hello from the base
```

The second line is missing[cite: 102]. Nothing crashes, and no error is printed: the second call simply prints an empty string[cite: 102]. Understanding exactly why this happens is the goal of Unit 1, and fixing it is the first objective in Unit 2[cite: 102].

By the end of the week, the fully implemented program will print four distinct lines, each serving as evidence that a specific piece of the low-level architecture is in place[cite: 102]:

```
main: privileged hello
main: this text lives in .data
SVC_Handler: privileged code printing on behalf of unprivileged code
BusFault_Handler: a bus fault happened
```

Practical Note: QEMU does not exit automatically when the program finishes because the bare-metal image enters an infinite spin loop (there is nowhere to return to)[cite: 102]. It must be manually stopped using `Ctrl-C`[cite: 102].

1.2 Memory Technologies in Embedded Systems

When writing a C program on a standard laptop, developers rarely think about *which* memory their variables reside in. On a microcontroller, this is a critical consideration because the chip contains two very different kinds of memory, and each piece of the program can live in only one of them[cite: 102].

1.2.1 SRAM (Static RAM): The Working Memory

The first kind of memory is **SRAM** (Static RAM)[cite: 102].

- **Direct Bus Connection:** SRAM is connected directly to the CPU's memory bus[cite: 102]. Every byte in it has a specific address, and the CPU reads or writes any byte using ordinary load or store instructions (the same instructions the compiler uses for `x = y;`)[cite: 102].
- **Speed:** It is extremely fast; on a microcontroller, one access typically takes about one clock cycle[cite: 102].

- **Writeability:** You can write to a single byte at any time, as often as you like[cite: 102].

- **Volatility:** Its primary weakness is that it is **volatile**: when power goes off, its content is completely lost[cite: 102]. Furthermore, when power comes back on, the content is not zero; it is **undefined**, consisting of a random pattern of bits left by the physical state of the memory cells[cite: 102].

Because of these characteristics, SRAM is the designated location for everything that *changes* while the program runs, specifically your variables and the stack[cite: 102].

1.2.2 NOR Flash: Execute-in-Place Memory

The second kind of memory is **NOR flash**. NOR flash sits on the memory bus, similar to a ROM: every word has a specific address, allowing the CPU to read any of them quickly and in any order. Because of this, the CPU can fetch instructions directly from the flash and execute them right where they are stored, without needing to copy the program into RAM first. This capability is known as **execute-in-place** (XIP).

NOR flash is **non-volatile**, meaning your program remains intact even after the board is unplugged or powered off. The price for these features is paid when writing data. Writing to flash is not a simple store instruction. It requires a two-step process:

1. An entire **sector** (a block of a few kilobytes) must be **erased**, setting every bit within it to 1.

2. An individual words can be **programmed**, which only turns bits from 1 to 0.

Both operations take microseconds to milliseconds and require a special command sequence sent to the flash controller. Consequently, for a running program, NOR flash is effectively **read-only**.

1.2.3 Contrasting with NAND Flash and Block Devices

It is worth contrasting this with the storage technologies commonly found in standard laptops, such as **NAND flash** (used in SD cards and SSDs) and hard disks.

These storage devices are **not** on the memory bus. They are accessed through a controller, one **block** (or page) at a time (typically 512 bytes to 4 KB). A block has no CPU address. The CPU cannot fetch an instruction from it, nor can it read a single variable directly: the contents must first be copied into RAM and then used from there.

This is exactly what a desktop operating system does when you start a program: the OS **loader** copies the executable from the SSD into RAM before running it. A microcontroller, however, has no loader. Its program already sits in NOR flash, at the exact address it runs from.

Technology	On the bus?	Execute from it?	Write at run time?	Keeps data at po
SRAM	yes	yes	yes, bytes	no
NOR flash	yes	yes (XIP)	no (erase/program)	yes
NAND / disk	no (blocks)	no	through a driver	yes

1.2.4 Applying Constraints to C Code

Put these facts side by side, and you get the rules for a C program on a microcontroller:

Based on these constraints:

- **Code and Constants:** Code stays in flash and runs directly from there. Constants (anything declared `const`, and string literals such as `"hello"`) also stay in flash and are read in place.

- **Uninitialized Variables:** Variables must live in RAM because they change.

- **Initialized Variables:** The interesting case is an **initialized** variable such as `int x = 5;`. The variable lives in RAM, but RAM is random at power-on. Therefore, the value `5` has to come from somewhere that survives power-off (flash). So the `5` is **stored in flash**, and something must **copy it into RAM at boot**, before `main()` runs and reads `x`. This missing copy mechanism is the whole reason the base program prints only one line.

1.2.5 The Linker Script and the Memory Map

The CPU physically sees a single address space of 4 GB (using 32-bit addresses)[cite: 8]. On every Cortex-M chip, this space is rigidly divided into regions whose purpose is fixed by the ARM architecture[cite: 8]. Our emulated board provides specific hardware at specific addresses: 4 MB of executable memory (FLASH) starting at address `0x0`, and 4 MB of working memory (RAM) starting at `0x20000000`[cite: 8].

The **Linker Script** (`.ld` file) is the architectural blueprint that tells the compiler tools exactly where these physical regions are and how to place the C program into them. Let us analyze the complete script provided for our board:

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M  /* code + read-only d
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M  /* data, bss, stack *
}

ENTRY(Reset_Handler)

_estack = ORIGIN(RAM) + LENGTH(RAM);    /* initial SP = top of RAM */
_Min_Stack_Size = 0x1000;               /* 4 KB reserved for stack */
_Min_Heap_Size = 0x1000;                /* 4 KB reserved for heap */
```

MEMORY: Defining the Regions

In the `MEMORY` block, each line explicitly describes one physical region[cite: 9]: a name you choose (`FLASH`, `RAM`), a start address (`ORIGIN`), a size (`LENGTH`), and access attributes (like `r` for read, `x` for execute)[cite: 9].

The linker actively uses the `LENGTH` values to protect your program: if you ask it to place more code than the flash memory can hold, the link intentionally fails at build time with an overflow error rather than producing a broken image that crashes at runtime[cite: 9, 15].

Immediately following the regions, the script calculates the `_estack` symbol[cite: 9]. This value (`0x20400000`) represents exactly one byte past the last available address in the RAM[cite: 9]. Because the Cortex-M stack physically grows downwards (toward lower memory addresses), initializing the stack pointer at the absolute top of the RAM gives it the maximum possible room to grow[cite: 9].

1.2.6 SECTIONS: Building the Executable Image

The `SECTIONS` block dictates exactly how the raw code and variables from your C files (the input sections) are packed into the final executable image (the output sections)[cite: 9].

```
SECTIONS
{
    /* 1. Vector table first, at address 0 */
    .isr_vector :
    {
        . = ALIGN(4);              /* word-align */
        KEEP(*(.isr_vector))       /* never garbage-collect it */
        . = ALIGN(4);
    } > FLASH

    /* 2. Executable code and read-only data */
    .text :
    {
        . = ALIGN(4);
        *(.text*)                  /* all executable code */
        *(.rodata*)                /* string literals, const variables */
        . = ALIGN(4);
    } > FLASH
```

The rules are strictly applied in top-to-bottom order[cite: 9]. The single dot (`.`) is the **Location Counter**: it represents the address the linker is currently at while placing bytes[cite: 10].

The `.isr_vector` section is placed into `FLASH` first, meaning it lands perfectly at address `0x0`, exactly where the hardware will look for it at boot[cite: 10]. The executable code (`.text`) and read-only constants (`.rodata`) are immediately packed after it, also in `FLASH`[cite: 9, 10].

Run Address and Load Address: The `.data` Section

The `.data` section, which holds initialized global variables (like our `data_message` string), introduces a critical architectural challenge[cite: 10]:

```
_sidata = LOADADDR(.data);  /* FLASH address of the stored initial va

/* 3. Initialized variables */
.data :
{
    . = ALIGN(4);
    _sdata = .;             /* start of .data in RAM */
    *(.data*)               /* initialized variables */
    . = ALIGN(4);
    _edata = .;             /* end of .data in RAM */
} > RAM AT > FLASH          /* run address in RAM, load address in FL
```

Every output section possesses two distinct memory addresses[cite: 10]:

- **VMA (Virtual/Run Memory Address):** Where the program expects the section to be when it runs[cite: 10].
- **LMA (Load Memory Address):** Where the section's raw bytes are actually stored in the physical flash image[cite: 10].

For code (`.text`), the VMA and LMA are identical because the CPU executes code directly from flash. For variables (`.data`), they *must* differ: variables must physically live in RAM so they can be written to, but their initial values must be permanently stored in FLASH so they survive power-off[cite: 10].

The directive `> RAM AT > FLASH` enforces exactly this: run address in RAM, load address in FLASH[cite: 10]. The script defines symbols (`_sidata`, `_sdata`, `_edata`) to record the exact boundary addresses so our assembly boot code knows exactly where to copy the bytes from and to[cite: 10].

bss, Heap, and Stack: RAM With No Stored Image

The final regions defined in the script exist purely in RAM and require zero physical storage space in the executable image[cite: 11]:

```
/* 4. Zero-initialized variables */
.bss :
{
    . = ALIGN(4);
    _sbss = .;              /* start of .bss */
    *(.bss*)                /* zero-initialized variables */
    *(COMMON)
    . = ALIGN(4);
    _ebss = .;              /* end of .bss */
} > RAM

/* 5. Heap region */
.heap :
```

```
    {
        . = ALIGN(8);
        _heap_bottom = .;
        . = . + _Min_Heap_Size; /* advance location counter by 4 KB */
        _heap_top = .;
    } > RAM

    /* 6. Link-time guard */
    __StackLimit = _estack - _Min_Stack_Size;
    ASSERT(__StackLimit >= _heap_top, "region RAM overflowed...")
}
```

- **The `.bss` section:** Holds global variables that explicitly start at zero (e.g., `int counter;`)[cite: 11]. There is no point storing thousands of zeros in the Flash image, so `.bss` has no LMA[cite: 11]. The boot code must actively fill this RAM range with zeros at runtime[cite: 11].
- **The Heap and Stack:** These are not standard input sections[cite: 11]. The script simply advances the location counter (`. = . + _Min_Heap_Size;`) to carve out a 4 KB chunk of empty RAM for the heap[cite: 11].
- **Safety Asserts:** The script dynamically calculates `__StackLimit` and ends with an `ASSERT` directive[cite: 11]. This prevents catastrophic memory corruption by intentionally failing the build if the calculated top of the heap collides with the bottom of the stack reservation[cite: 11].

1.2.7 Linker-Defined Symbols Are Addresses, Not Variables

Symbols defined in the linker script such as `_sdata`, `_edata`, and `_estack` are unique: **they have no physical storage**[cite: 11].

There is no physical memory cell called `_sdata` holding a value[cite: 11]. The linker simply assigns each symbol an address, and that address *itself* is the information[cite: 11]. This distinction is critical when referencing them from C:

```
extern uint32_t _sdata;
uint32_t start = (uint32_t)&_sdata; /* OK: gets the actual address */
uint32_t oops = _sdata;             /* WRONG: reads garbage from RAM */
```

In assembly, the boot code utilizes these symbols strictly as mathematical boundaries (e.g., looping from `_sidata` to `_sdata`) to initialize the C runtime[cite: 11].

1.2.8 Reading the Result: The Map File, `nm`, and `objdump`

You now know what the linker script asks for. You can verify what the linker and compiler actually did using standard toolchain utilities:

- **The Map File (`.map`):** Generated by the linker, it repeats your `MEMORY` regions and lists every output section, its size, and the load address for `.data`.
- **`arm-none-eabi-nm -n`:** Lists all symbols inside an object file (`.o`) or final executable (`.axf`), sorted by address. The letter before each name identifies its kind:

  - **T / t:** Code (`.text`; uppercase means global, lowercase means local/static).
  - **R:** Read-only data (`.rodata` or constants).
  - **D:** Initialized data destined for RAM (`.data`).
  - **B:** Uninitialized data destined for RAM (`.bss`).
  - **U:** Undefined symbol (an external reference waiting for the linker to resolve it).

- `arm-none-eabi-objdump -h:` Prints one line per output section, revealing its exact size, its VMA (run address), and its LMA (load address).

Analyzing the Map File (`.map`)

When the linker successfully finishes linking the object files, it produces an `output.map` file. This file is the absolute source of truth for understanding exactly where every single byte of your program physically landed in memory.

Looking at the generated `output.map` for our base skeleton, we can observe the Linker strictly enforcing our rules:

```
Memory Configuration
Name             Origin             Length             Attributes
FLASH            0x00000000         0x00400000         xr
RAM              0x20000000         0x00400000         xrw
```

The `Memory Configuration` block confirms that the physical hardware map was understood exactly as we defined it.

As we scroll down into the `Linker script and memory map` section, we see the individual output sections being populated:

```
.isr_vector     0x00000000        0x8
 *(.isr_vector)
 .isr_vector    0x00000000        0x8 build/boot/startup.o
                0x00000000              isr_vector

.text           0x00000008       0x58
 *(.text*)
 .text          0x00000008        0x6 build/boot/startup.o
                0x00000008              Reset_Handler
 .text          0x00000010       0x34 build/src/main.o
                0x00000028              main
```

This section clearly demonstrates the "folding" behavior:

- The vector table (`.isr_vector`) was correctly placed at address `0x00000000`.
- Immediately after it, at address `0x00000008`, the linker began packing the executable code (`.text`).
- It first placed `Reset_Handler` from `startup.o`, and then placed the `main` function from `main.o` starting at `0x00000028`.

Finally, analyzing the `.data` section reveals the LMA/VMA split perfectly:

```
                0x00000060                _sidata = LOADADDR (.data)
...
.data           0x20000000        0x20 load address 0x00000060
                0x20000000                _sdata = .
 .data          0x20000000        0x20 build/src/main.o
                0x20000000                data_message
```

Here, the linker explicitly proves that our initialized array `data_message` is mapped to run from RAM at address `0x20000000` (its VMA), but its physical load address (LMA) in the image is `0x00000060`, exactly as configured via `LOADADDR`.

Analyzing the Final Executable (`.axf`)

Once the linker has stitched all the object files together using the rules defined in the Linker Script, we can inspect the final executable image (`scratch-os.axf`) using `nm`:

```
$ arm-none-eabi-nm scratch-os.axf
00000008 T Reset_Handler
00001000 A _Min_Heap_Size
00001000 A _Min_Stack_Size
203ff000 A __StackLimit
20000020 B _ebss
20000020 D _edata
20400000 R _estack
00000060 T _etext
20000020 B _heap_bottom
20001020 B _heap_top
20000020 B _sbss
20000000 D _sdata
00000060 A _sidata
20000000 D data_message
0000000c t hang
00000000 R isr_vector
00000028 T main
00000010 t semihost_write0
```

This final output perfectly validates our entire architecture:

- **Resolved Symbols:** The previously undefined (**U**) symbols from the object files have been assigned exact physical addresses. `main` is now located at `0x00000028`, and `Reset_Handler` is at `0x00000008` (both marked **T** for executable text).
- **Vector Table:** `isr_vector` is perfectly anchored at `0x00000000` (**R** for read-only).
- **Variables in RAM:** Our writable global array, `data_message`, sits precisely at `0x20000000` (**D** for initialized data), which perfectly matches the linker-defined symbol `_sdata`.

- **Absolute Linker Symbols:** Symbols defined purely by math in the linker script (like `_Min_Stack_Size` or `_sidata`) are tagged as **A** (Absolute). They do not belong to any specific code section; they are simply hardcoded constant addresses used by our assembly boot loop.

Analyzing the Object Files (`.o`)

Before the linker merges everything into the final image, we can inspect the raw object files generated by the compiler and assembler using `nm`.

If we inspect the compiled assembly boot code (`startup.o`), we see:

```
$ arm-none-eabi-nm startup.o
00000000 T Reset_Handler
         U _estack
00000004 t hang
00000000 R isr_vector
         U main
```

This output perfectly validates our code:

- `Reset_Handler` is global code (**T**), while `hang` is a local assembly label (**t**).
- `isr_vector` is treated as read-only data (**R**) destined for FLASH.
- `_estack` and `main` are marked as **U** (Undefined). The assembler encountered these names but did not find them in `boot.s`. It leaves them blank, trusting the Linker Script and `main.o` to provide their actual addresses later.

Similarly, if we inspect our C program (`main.o`):

```
$ arm-none-eabi-nm main.o
00000000 D data_message
00000018 T main
00000000 t semihost_write0
```

Here, the C compiler correctly categorized our components:

- `main` is a globally visible C function (**T**).
- `semihost_write0` is a `static` C function, making it strictly local to this file (**t**).
- `data_message` is an initialized global array, so the compiler explicitly tagged it for the `.data` section (**D**).

When the linker runs, it will stitch `startup.o` and `main.o` together, replacing the **U** symbols with the real addresses assigned during the layout phase.

Pointers, Local, and Static Variables

Pointers deserve a moment of care, because a pointer is itself a variable[cite: 109]. In the declaration `const char *p = "hi";`, the characters `"hi"` are a string literal and go to `.rodata`; however, `p`, the variable holding their address, is an initialized writable variable and goes to `.data`[cite: 109].

This table of section rules only applies to declarations made **outside any function**[cite: 109]. A local variable inside a function has no section at all: it lives on the stack while the function runs, and it is gone as soon as the function returns[cite: 109].

Try It: Testing Global Declarations

You can test these placement rules by adding the following **global** variables to `src/main.c`, right after the line that defines `data_message` (before `main`), so outside any function[cite: 109]:

```
const int limit = 100;
int counter = 5;
int total;
int zero = 0;
const char *greeting = "hi";
char buffer[64];
```

Write down your prediction for each one, then check[cite: 109]:

```
make
arm-none-eabi-nm build/src/main.o
arm-none-eabi-nm -n build/scratch-os.axf
```

You find `counter` and `greeting` marked `D` (initialized data), and `total`, `zero`, and `buffer` marked `B` (bss)[cite: 109]. The variable `limit` is the interesting one: in `main.o` it is marked `R` (read-only data), but in the final image it is marked `T`, as if it were code[cite: 109, 110]. The reason is the linker script: it puts `.rodata` *inside* the `.text` output section, and in the final image `nm` classifies a symbol by the output section it ended up in[cite: 110].

The string `"hi"` has no name, so `nm` does not show it, but the map file shows the `.rodata` input sections of `main.o` inside `.text`[cite: 110]. You can find its bytes directly by dumping the contents of the `.text` output section[cite: 110]:

```
$ arm-none-eabi-objdump -s -j .text build/scratch-os.axf
 ...
 0038 00bffde7 4c000000 00000020 64000000  ....L.......  d...
 0048 68690000 6d61696e 3a206865 6c6c6f20  hi..main: hello
```

Each line starts with the address of its first byte, shows 16 bytes, and repeats the same bytes as text on the right. At address `0x44` you find `64 00 00 00`: that is `limit`, whose value is 100 (`0x64`)[cite: 110]. At `0x48` you find `68 69 00`, the characters `h` and `i` followed by the terminating zero: the string `"hi"` starts at address `0x48`, in FLASH[cite: 110].

Pointers in RAM and Little-Endianness

How does the program find it? Through `greeting`, which is a pointer stored in RAM. Dump `.data`[cite: 110]:

```
$ arm-none-eabi-objdump -s -j .data build/scratch-os.axf
 20000000 6d61696e 3a207468 69732074 65787420  main: this text
 20000010 6c697665 7320696e 202e6461 74610a00  lives in .data..
 20000020 05000000 48000000                    ....H...
```

Because you added the variables right after `data_message` in `src/main.c`, the first two lines, from `0x20000000` to `0x2000001F`, are `data_message` (you can read its text on the right), and your variables follow: `counter` at `0x20000020` and `greeting` at `0x20000024`[cite: 110]. The compiler places variables in `.data` in the order you define them.

To read the numbers you need one more fact: the Cortex-M3 is **little-endian**, which means that a 32-bit number is stored with its lowest byte first[cite: 110].

- The four bytes `05 00 00 00` are the number 5 (the value of `counter`)[cite: 110].
- The four bytes `48 00 00 00` are the number `0x00000048`[cite: 110]. That is the value of `greeting`, and it is exactly the FLASH address where `"hi"` starts[cite: 110].

This is all a pointer is: a variable whose value is an address[cite: 110].

Local vs. Static Local Variables

You can see the local-vs-global contrast directly. Inside `main`, add `int local = 7;` and `static int s = 3;`, then check `nm` again[cite: 109].

- `local` never appears at all: it is not a global, it has no fixed address, and it only exists on the stack while `main` is running[cite: 109].
- `s` does appear, as `d s.0`: lowercase `d` means it lives in `.data` but is visible only inside this file, and the compiler appends a `.0` (or `.N` suffix) to tell one function's static local apart from another's if they share a name[cite: 109]. Unlike `local`, a `static` local keeps one fixed address for the whole run of the program, exactly like a global — it is only its *name* that stays private to the function[cite: 109].

1.2.9 Advanced Memory Layout: Custom Sections and Orphans

You are not strictly limited to the default sections the compiler chooses[cite: 110]. GCC provides compiler attributes that allow you to explicitly name the input section of a variable or a function[cite: 110]:

```
int tagged __attribute__((section(".mysection"))) = 42;
```

Where that custom section physically ends up is entirely up to the linker script, **if the script names it**[cite: 110]. A section that does not match any rule in the `SECTIONS` block is called an **orphan**[cite: 110]. The linker does not reject an orphan section; instead, it places it by guessing based on its flags (e.g., whether it is writable or executable) and places it adjacent to sections with similar properties[cite: 110].

While this guess usually allows the program to link successfully, the orphan section is placed completely outside your control and, critically, outside any predefined symbol ranges[cite: 110].

If you add the `tagged` variable to the base project and examine the output using `arm-none-eabi-nm -n`, you will see the linker symbols placed next to your variable[cite: 110]:

```
20000000 D _sdata
20000020 D _edata
20000020 D tagged
```

A new output section `.mysection` appears. The linker saw a writable, initialized section, so it placed it in RAM **right after** `.data`, and gave it its own load address in FLASH.

Now compare its address with `_edata`: `tagged` sits exactly at `_edata`, that is, *after* the end of the range `_sdata`...`_edata`. A boot loop that copies only `_sdata`...`_edata` will never copy it, so `tagged` will not hold 42 when `main()` runs. The program links without a single warning and is still wrong. That is the fundamental danger of orphans.

If you declare a `const int` in a section called `.myconst` instead, the linker sees a read-only section and places it safely in FLASH, between `.isr_vector` and `.text`.

1.2.10 Overflowing a Region

The `LENGTH` values in the `MEMORY` block are not decoration: the linker actively enforces them. First, measure how much flash the base image needs:

```
arm-none-eabi-size build/scratch-os.axf
```

The FLASH region has to hold `text + data`: the code and constants, plus the stored copy of `.data`'s initial values. For the base skeleton, this is $96 + 32 = 128$ bytes.

If you edit `scripts/mps2_m3.ld` and set FLASH `LENGTH = 100`, you can test this enforcement. Rebuild with `make -B` (the `Makefile` does not automatically list the linker script as a prerequisite, so `-B` forces everything to be rebuilt). The link will intentionally fail with:

```
region 'FLASH' overflowed by 28 bytes
```

$128 - 100$ is exactly 28. If you set `LENGTH = 128`, the link succeeds because the image fits exactly.

If you then restore FLASH and set RAM `LENGTH = 8K` instead, `_estack` moves down to `0x20002000`. The 4 KB stack reservation starts 4 KB below it, meaning the 4 KB heap no longer fits between `.bss` and the stack. The build failure in this scenario comes directly from the script's own `ASSERT` line, printing its custom message about the heap colliding with the stack.

1.3 Reset: The Hardware Contract and `boot.s`

Once the memory layout is finalized in the linker script and loaded into memory, the machine can be powered on. The next question is how the CPU starts using it[cite: 16].

When the Cortex-M3 comes out of reset, before any of your code runs, there is absolutely no C runtime available[cite: 16]. The '.data' section is not copied to RAM yet, and '.bss' is not zeroed. To boot the system, the hardware automatically executes a strictly predefined sequence. It does exactly two things[cite: 16]:

1. It reads the 32-bit word at address '0x0' and puts it directly into the Stack Pointer ('SP')[cite: 16].

2. It reads the word at address '0x4' and puts it directly into the Program Counter ('PC')[cite: 16]. Execution starts at that exact address.

Everything else (copying '.data', zeroing '.bss', configuring the CPU, calling 'main()') is entirely the responsibility of your bare-metal code[cite: 16]. This foundational runtime setup is written entirely in assembly in a file named `boot/startup.s`.

1.3.1 The Assembly Environment

Before writing the actual instructions, the assembly file configures the build tools using assembler directives:

- `.syntax unified`: Instructs the assembler to use modern ARM syntax[cite: 23].

- `.cpu cortex-m3`: Ensures the assembler only accepts instructions physically present on the Cortex-M3, catching typos at compile-time.

- `.thumb`: Emits Thumb instructions. Cortex-M CPUs can *only* execute Thumb code; they lack a standard 32-bit ARM mode.

Any symbol the assembler does not find defined locally (like `_estack` from the linker script, or `main` from `main.c`) is automatically treated as an external reference, leaving it up to the linker to fill in the correct addresses.

1.3.2 The Vector Table

Those two initial words read at reset are the beginning of the **Vector Table**: an array of addresses at a fixed place in memory (address '0x00000000' at reset)[cite: 16].

The table exists because of exceptions[cite: 16]. An exception is an event that forces the CPU to stop its current task and run a specific piece of code called a handler[cite: 16]. Slot N of the table physically holds the memory address of the handler for exception N[cite: 16]. Slot 0 is unique: it is not executable code, but purely the initial value of the Stack Pointer[cite: 16].

In `boot/startup.s`, we construct this table explicitly. Notice how the assembly mirrors standard C array initialization:

```
    .section .isr_vector,"a",%progbits
    .global isr_vector
    .type isr_vector, %object

isr_vector:
    .word _estack                   /* 0: initial SP = top of RAM */
    .word Reset_Handler             /* 1: reset: first code after power-on */
```

C Equivalent:

```
void (* const isr_vector[])(void) = {
    (void (*)(void)) &_estack,  /* Slot 0: Initial Stack Pointer */
    Reset_Handler,              /* Slot 1: Reset execution address */
};
```

Directive breakdown: The `.section .isr_vector,"a",%progbits` directive explicitly tells the tools to place these words into a section named `.isr_vector`. The linker script ensures this specific section is always physically placed first in the FLASH memory, guaranteeing it lands precisely at address '0x0'.

The Thumb Bit Constraint

One critical detail you will observe in a debugger: the value stored in word 1 is not exactly the raw address of 'Reset_Handler', but that address with its lowest bit set to 1.

Instructions always sit at even addresses (2-byte or 4-byte aligned), so bit 0 of a code address is physically unused[cite: 16]. ARM repurposes bit 0 as a flag to mean "this is Thumb code"[cite: 16]. Because the Cortex-M3 executes only the Thumb instruction set, the hardware strictly requires this bit to be 1[cite: 16].

KEEP and Garbage Collection

The linker possesses an optional optimization feature known as section garbage collection[cite: 16]. It drops every section of code or data that no other software actively references[cite: 16].

Because no software code actively calls or reads `isr_vector` (only the hardware reads it at power-on), the garbage collector would consider the table useless and throw it away, causing an immediate hardware lockup at boot[cite: 16, 17]. The `KEEP (*(.isr_vector))` directive in the linker script explicitly forces the linker to preserve the table regardless of software references[cite: 16, 17].

1.3.3 The Reset Handler

Once the vector table is read, execution officially begins at the 'Reset_Handler'. This is standard executable code, placed into the `.text` section.

```
    .section .text,"ax",%progbits
    .global Reset_Handler
    .type Reset_Handler, %function
    .thumb_func
Reset_Handler:
    /* Runtime setup steps (stack, data, bss) go here. */
```

```
    bl main                     /* Branch with Link to C code */
hang:
    b hang                      /* Infinite loop if main returns */

    .ltorg                      /* Literal pool placement */
```

C Equivalent:

```
void Reset_Handler(void) {
    /* Setup runtime... */
    main();
    for (;;) { }
}
```

Directive breakdown:

- **.thumb_func**: Instructs the tools that the immediately following label ('Reset_Handler') is a Thumb function. When the assembler builds the vector table above, it sees this tag and automatically sets bit 0 to 1, fulfilling the hardware's Thumb bit constraint.
- **.ltorg**: Generates the **Literal Pool**. ARM instructions are only 16 or 32 bits wide, meaning they cannot physically contain a full 32-bit constant (like the memory address of *_estack*) inside the instruction itself. The assembler stores the actual 32-bit constant in a nearby table (the literal pool) and translates your instruction into a PC-relative load. *.ltorg* guarantees this table of constants is placed safely after the last instruction, where the CPU will never accidentally attempt to execute it as code.

1.3.4 Back to the Base: Why Only One Line Prints

With the memory layout and hardware contract fully mapped, we can finally explain the behavior of the base skeleton observed at the beginning of the chapter[cite: 18].

If you run the base code (`make run`), the `main()` function attempts to print two lines:

1. A standard string literal[cite: 18].
2. `data_message`, an initialized global array[cite: 18].

Let us look closely at how these are defined in `src/main.c`, as it perfectly illustrates the memory concepts from Unit 1:

```
/* A global array with an initial value.
 * Because it is writable data, it MUST live in RAM. */
char data_message[] = "main: this text lives in .data\n";

int main(void)
{
    /* 1. Printing a string literal */
    semihost_write0("main: hello from the base\n");
```

```
    /* 2. Printing the global array */
    semihost_write0(data_message);

    for (;;) { }
}
```

When `main()` executes, here is exactly what happens at the hardware level:

- **The String Literal (Line 1):** A string literal like `"main: hello..."` is read-only `const` data. The compiler naturally places it in the `.rodata` section, which the linker script puts directly into FLASH memory alongside the executable code. It needs no copying to be read: it is already sitting at the physical address it will be read from, right from the very first instruction. Therefore, this line always prints correctly[cite: 18].
- **The Global Array (Line 2):** Unlike a string literal, a global array is *writable* data, so it must physically live in RAM (since FLASH cannot be written to at runtime). As confirmed by the linker script, the `.data` section has its VMA (Run Address) at `0x20000000` in RAM, but its initial bytes are stored at a FLASH LMA (Load Address)[cite: 18].

Until something physically copies its initial value from FLASH to RAM, this array's content in RAM is whatever happened to already be there at power-on. In our base skeleton, `Reset_Handler` contains no code to move the bytes from the LMA to the VMA[cite: 18]. Consequently, the RAM at `0x20000000` in QEMU is filled with zeros, which C interprets as an empty string[cite: 18]. Thus, the second call prints nothing[cite: 18].

Writing the assembly loop to perform this missing memory copy is the very first task in building our C runtime.

1.4 Taking Control of the CPU

In this unit, you rebuild `Reset_Handler` in `boot/startup.s` yourself, in assembly, one step at a time. To do that, you need two things you have not used before: a debugger to *see* what the CPU does, and enough ARM assembly to read and write the boot code.

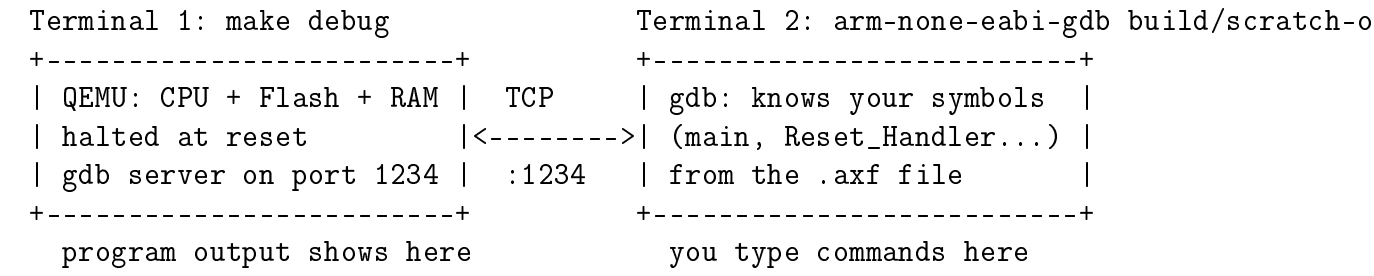
1.4.1 What is a Debugger?

A **debugger** is a program that controls another program while it runs. It can **stop** the program at any instruction you choose (a **breakpoint**) or right at the start, before anything has run. It can **step** it forward one machine instruction, or one C line, at a time. While the program is stopped, it can **inspect** the CPU registers and any memory address, and even change them. And it can **continue** the program until the next breakpoint.

On a desktop, a crashing program at least prints an error message. On a microcontroller, there is often no console at all, and a crash simply looks like "nothing happens". A debugger is frequently the *only* way to see what the CPU is really doing. The standard tool is **gdb**, the GNU debugger, part of the same toolchain family as gcc. The ARM version you use is `arm-none-eabi-gdb`; on Linux, the gdb that understands ARM is usually called `gdb-multiarch` instead.

1.4.2 A Remote Debugger: Two Programs, Two Terminals

In this setup, the program being debugged does not run on the same "machine" as gdb. It runs inside QEMU, which emulates the Cortex-M3 board on your laptop. gdb talks to QEMU over a network connection (a TCP socket on your own machine). So you use two terminals:



In terminal 1, you start QEMU with `make debug`. The `debug` target in the `Makefile` runs the same QEMU command as `make run`, plus two options: `-S`, which freezes the emulated CPU before its very first instruction, and `-gdb tcp::1234`, which starts a gdb server that waits for a connection on TCP port 1234. Anything the program prints still appears in this terminal.

Reconstructing the Debug Target

The `debug` target is therefore nothing more than a shorthand, and you can reconstruct it for any image. Take the QEMU command from the `run` target of the `Makefile` you are working with, and append `-S -gdb tcp::1234`:

```
qemu-system-arm -machine mps2-an385 -monitor null -semihosting \
    -semihosting-config enable=on,target=native \
    -kernel build/scratch-os.axf -serial stdio -nographic \
    -S -gdb tcp::1234
```

This matters after this week: the later checkpoints' Makefiles have a `run` target but no `debug` target, and this command, run from the checkpoint's folder after `make`, is how you debug them.

In terminal 2, you start gdb and give it the ELF file, `arm-none-eabi-gdb build/scratch-os.axf`. gdb reads the ELF only for its **symbols** and debug information, so that it knows the addresses of `main`, `Reset_Handler`, `data_message`, and so on. It does not load the code: QEMU already holds the program. You then connect with `target remote :1234`.

The GDB Commands You Need

This small set of commands is enough for everything this week:

1.4.3 Attaching at Reset

The first thing to do with the debugger is to catch the CPU *before* its first instruction, and check the reset contract from Unit 1. In the base folder:

```
make debug                                  # terminal 1: QEMU waits
arm-none-eabi-gdb build/scratch-os.axf      # terminal 2
```

Command	What it does
`target remote :1234`	connect to the gdb server (QEMU) on this machine
`info registers sp pc`	show some registers (`info registers` alone: all of them)
`x/2wx 0`	**x**amine memory: 2 **w**ords (32 bits each) in he**x**, at address 0
`x/s &data_message`	examine memory as a C **s**tring
`p/x $r1`	**p**rint one register (or a C expression) in hex
`display/i $pc`	after every step, show the next **i**nstruction
`stepi` (`si`)	execute exactly one machine instruction (pressing Enter repeats it)
`break main` (`b`)	stop when execution reaches `main` (or any label)
`continue` (`c`)	run until the next breakpoint; `Ctrl-C` interrupts it
`quit` (`q`)	leave gdb (QEMU keeps running: stop it with `Ctrl-C`)

and in gdb:

```
(gdb) target remote :1234
(gdb) info registers sp pc
(gdb) x/2wx 0
```

You see `sp` equal to `0x20400000`, `pc` equal to `0x8 <Reset_Handler>`, and at address 0 the two words `0x20400000 0x00000009`. **No instruction has run yet**, yet SP and PC already have values: the CPU itself copied vector word 0 into SP and vector word 1 into PC.

Compare the two: word 1 is `0x9`, PC is `0x8`. This is the Thumb bit from Unit 1. The CPU uses bit 0 to check that the target is Thumb code and then clears it; the instruction really starts at `0x8`. And why `0x8` in the first place? The base's vector table has only two words (8 bytes), and the code is placed right after it in FLASH.

1.4.4 Cortex-M3 Programmer's Model: The Registers

To follow the CPU instruction by instruction, you need to know what it holds. The **programmer's model** is the set of registers your code, and gdb, can see. All of them are 32 bits wide.

Register	Name	Role
R0–R12	general purpose	your values; a C function call passes its first arguments in R0–R3
R13	**SP**	stack pointer: address of the top of the stack
R14	**LR**	link register: where to return after a function call
R15	**PC**	program counter: address of the next instruction
—	**xPSR**	status: flags, Thumb bit, current exception

At this level, there are no variables: only these registers and memory. When your C code says `x = x + 1`, the compiler turns it into "load `x` from memory into a register, add 1 to the register, store the register back". The stack, as you know, grows **downwards**: pushing a value first does `SP = SP - 4`, then writes the value at the new SP.

The Status Register (xPSR) and CPU Modes

xPSR (program status register) packs three critical pieces of information into one register:

- **Flags (Bits 31–28):** The **N Z C V** flags (negative, zero, carry, overflow), set by comparison instructions and used by conditional jumps.
- **Thumb Bit (Bit 24):** The **T** state bit, which must always be 1 on Cortex-M (this is where bit 0 of the vector entry ends up).
- **Exception Number (Bits 8–0):** Holds the number of the exception currently being handled, or 0 if none.

That last field dictates which of the two **modes** the CPU is in:

- **Thread mode:** The CPU runs normal code (`Reset_Handler`, `main()`, and the functions they call); the exception number is 0.
- **Handler mode:** The CPU runs an exception handler, and the field holds that exception's number.

Note: Code in Thread mode can also be privileged or unprivileged; everything in this specific unit runs privileged, and Unit 3 explains the difference.

1.4.5 ARM Assembly Basics: A Load/Store Architecture

The boot code has to be written in assembly because, before it runs, there is no valid stack and no initialized data, so C code cannot yet be trusted. You need only a handful of instructions, and each has a direct C equivalent.

A load/store machine: ARM arithmetic works only on registers. Memory is touched only by **load** instructions (`ldr`, memory to register) and **store** instructions (`str`, register to memory).

Assembly	C equivalent
`ldr r3, [r0]`	`r3 = *r0;`
`str r3, [r1]`	`*r1 = r3;`
`ldr r3, [r0], #4`	`r3 = *r0; r0 += 4;` (that is, `r3 = *src++;`)
`str r3, [r1], #4`	`*r1 = r3; r1 += 4;` (that is, `*dst++ = r3;`)
`strb r1, [r0, #7]`	`((uint8_t *)r0)[7] = r1;` (one **b**yte, at offset 7)

The square brackets mean "the memory at this address". The form `[r0], #4` is called **post-increment**: the instruction first uses the address in `r0`, then adds 4 to `r0`. One word is 4 bytes, so a loop using this syntax walks through memory word by word, exactly like `*dst++` in C.

Constants, Labels, and Jumps

Moving data between registers and controlling program flow requires another set of basic instructions:

The 32-bit Instruction Encoding Constraint (Literal Pools): The `ldr rX, =value` form needs a word of explanation because you will see it look different in gdb. An instruction is only 16 or 32 bits long, so it mathematically cannot contain an arbitrary 32-bit constant alongside its opcode.

The assembler therefore stores the constant in a small table of constants near the code, called the **literal pool**, and turns your `ldr r0, =_estack` into "load r0 from

Assembly	C equivalent
`ldr r0, =_estack`	`r0 = (uint32_t)&_estack;` (any 32-bit value)
`mov sp, r0`	`sp = r0;` (register to register)
`movs r3, #0`	`r3 = 0;` (small constants only)
`copy_data:`	a **label**: a name for this address, like a C `goto` label
`b copy_data`	`goto copy_data;`
`bl main`	`main();`: **b**ranch with **l**ink, saves the return address in LR

that table". In gdb, this appears as something like `ldr r0, [pc, #92]` (an address relative to the PC).

The `.ltorg` directive at the end of `Reset_Handler` in `boot/startup.s` tells the assembler where to put this pool: typically after the last instruction, where the CPU will never try to execute it as code.

Control Flow: Decisions, Bits, and Interrupts

Branching and loops in assembly rely entirely on the `xPSR` status register. A decision is a two-step process: compare values to set condition flags, then jump based on those flags.

Assembly	C equivalent
`cmp r1, r2`	compute `r1 - r2`, keep only the flags N Z C V
`bhs done`	`if (r1 >= r2) goto done;` (**h**igher or **s**ame, unsigned)
`orr r1, r1, #0x10`	`r1 |= 0x10;`
`cpsid i`	stop accepting interrupts (`__disable_irq()` in vendor C code)
`cpsie i`	accept interrupts again (`__enable_irq()`)

With `cmp` and a conditional branch, you can build any loop. A C `while` loop becomes "test at the top, jump back at the bottom":

```
loop:   cmp r1, r2          @ while (r1 < r2) {
        bhs done            @
        ...                 @     body
        b   loop            @ }
done:
```

The loop continues while `r1 < r2`; as soon as `r1 >= r2`, `bhs` jumps out of the loop.

Assembler Directives

Lines that start with a dot (`.`) produce no instructions: they steer the assembler itself. The directives used in `boot/startup.s` are:

```
    .section .isr_vector,"a",%progbits   @ put what follows in section .i
    .global isr_vector                   @ make the name visible to the l
isr_vector:
    .word _estack                        @ one 32-bit value
    .word Reset_Handler

    .section .text,"ax",%progbits
```

```
    .global Reset_Handler
    .thumb_func                              @ next label is Thumb code: its
Reset_Handler:
```

The vector table block translates to the following C equivalent:

```
__attribute__((section(".isr_vector")))
const uint32_t isr_vector[] = { (uint32_t)&_estack, (uint32_t)Reset_Hand
```

- `.section`: Chooses the section, with flags. `"a"` means the section occupies space in the image; `"x"` means it is executable.
- `.global`: Makes a name visible to other files and to the linker. Without it, a label acts like a `static` name in C.
- `.word`: Allocates exactly one 32-bit word and stores the given value inside it.
- `.thumb_func`: Dictates that the next label is Thumb code. This instructs the tools to store `Reset_Handler` with bit 0 set to 1, which is where the `0x9` in the base (and `0x41` in the solution) comes from.

The file also uses `.type` and `.size` to tell `gdb` and `nm` whether a name is code or data and its total size.

1.5 Writing the Startup Script (`Reset_Handler`)

The `Reset_Handler` in `boot/startup.s` has one critical job: to turn a bare CPU into an environment that can safely run C code.

Here is the complete sequence of operations, in the exact order the boot code executes them:

Step	**Why**
1. Mask interrupts	Nothing may interrupt a half-built system
2. Set SP	C functions inherently need a stack
3. Copy `.data` FLASH → RAM	Initialized global variables must get their values
4. Zero `.bss`	C promises uninitialized global variables start at 0
5. Configure the CPU	Set up vector table address, fault handlers, traps, priorities
6. Unmask interrupts, `bl main`	Hand execution over to C
7. Loop forever	If `main` ever returns, there is nowhere to go

In the base skeleton, `Reset_Handler` has only steps 6 and 7 (and without the unmask). The following sections go through building the remaining steps in order.

1.5.1 Steps 1 and 2: Mask Interrupts and Set the Stack

An **interrupt** is an exception raised by hardware (e.g., a timer expiring or a device receiving data): the CPU drops what it is doing and runs that interrupt's handler. During setup, absolutely nothing is ready to handle one: a handler written in C could attempt to read a variable that has not been initialized yet.

Therefore, the very first instruction, `cpsid i`, tells the CPU to explicitly ignore interrupts for now. (While no device has been told to raise an interrupt yet, writing it makes the rule explicit instead of relying on luck).

Next, the stack pointer (`SP`) must be set:

```
@ C equivalent: sp = &_estack;
ldr r0, =_estack     @ top of RAM, 0x20400000
mov sp, r0
```

Because C has no way to name SP directly, the C equivalent is pseudo-code. One might wonder why this step is strictly needed, since the hardware already loaded SP from word 0 of the vector table at reset. The answer is that `Reset_Handler` is not *always* reached through a real hardware reset: a custom bootloader or a debugger may jump to it directly, with SP holding an arbitrary or invalid value.

Setting SP explicitly makes the code robust and correct in every case, and demonstrates that the stack is nothing more than an address in a register. Notice the two-step ARM pattern: because SP is a special register, the address is first loaded into an ordinary general-purpose register (`r0`), and then explicitly copied into SP using `mov`.

1.5.2 Step 3: Copy `.data` from FLASH to RAM

This is the critical step that fixes the base skeleton's missing print line. The linker script gives you three key addresses:

```
FLASH:  [ .text | .rodata | initial values of .data ]
                             ^ _sidata
RAM:    [ .data                | .bss | ...        stack ]
          ^ _sdata             ^ _edata                  ^ _estack
```

The copy operation is a simple loop, which you can write first in C and then translate line by line into assembly:

C Equivalent	ARM Assembly

`uint32_t *src = &_sidata;`
`uint32_t *dst = &_sdata;`
	`ldr r0, =_sidata`
	`ldr r1, =_sdata`
	`ldr r2, =_edata`

`while (dst < &_edata)`	`copy_data: cmp r1, r2` `bhs copy_data_done`

`*dst++ = *src++;`	`ldr r3, [r0], #4`
	`str r3, [r1], #4`
	`b copy_data`

	`copy_data_done:`

Register `r0` plays the role of `src`, `r1` of `dst`, and `r2` holds the end address. The loop is named `copy_data` in `boot/startup.s`, and it exits to `copy_data_done`. It copies whole words, 4 bytes at a time; this is perfectly safe because the linker script intentionally aligns the start and end of `.data` to multiples of 4.

If you write steps 1–3 into the base skeleton (just before `bl main`) and run it, the missing line finally appears:

```
make run
main: hello from the base
main: this text lives in .data
```

Before the copy, `data_message` sat in RAM as all zeros and printed as an empty string; now it successfully holds its initial value.

Watching the Copy Loop in GDB

It is worth watching your loop run because it makes the abstract ideas of LMA and VMA concrete. Start `make debug` in terminal 1 and gdb in terminal 2, then:

```
(gdb) target remote :1234
(gdb) display/i $pc
(gdb) stepi                          # press Enter to repeat, until "cmp r1,
(gdb) p/x $r0                        # src: a FLASH address
(gdb) p/x $r1                        # dst: 0x20000000, start of RAM
(gdb) x/s &data_message              # "" : nothing copied yet
(gdb) stepi 5                        # one full turn of the loop
(gdb) p/x $r1                        # 0x20000004
(gdb) break copy_data_done
(gdb) continue
(gdb) x/s &data_message              # "main: this text lives in .data\n"
```

As you step, `display/i $pc` shows each instruction just before it runs: `cpsid i`, then the `ldr` from the literal pool, then `mov sp, r0`, then the three `ldr` commands that load the loop bounds.

When you reach `cmp r1, r2`, `r0` holds a small flash address (e.g., `0x88`, depending on how much code precedes it), and `r1` holds `0x20000000`. `x/s &data_message` shows the empty string because nothing has been copied yet.

Five more `stepi` commands execute one full turn of the loop (`cmp`, `bhs`, `ldr`, `str`, `b`), and `r1` moves to `0x20000004` (one word copied). Instead of stepping 8 times more, you put a breakpoint on the label after the loop and continue. `r1` climbs until it equals `r2` (`&_edata`, which is `0x20000020`), and only then does `x/s &data_message` show the full string perfectly mapped in RAM.

Try It: Breaking the Stack on Purpose

Seeing a wrong SP fail is the best way to believe that SP matters. In `boot/startup.s`, inside `Reset_Handler`, find the line you wrote in Step 2 (`ldr r0, =_estack`) and intentionally replace `_estack` with a bad address. (Note that the value the CPU loaded from vector word 0 at reset is still the correct one; it is your `mov sp, r0` that forcefully overwrites it with the bad address).

Test 1: Address Outside of RAM (`0x30000000`) If you force SP to `0x30000000` and run `make run`, nothing is printed at all. QEMU stops with:

```
qemu: fatal: Lockup: can't escalate 3 to HardFault (current priority -1)
```

Here is what happened: the first thing `main` does is push registers onto the stack, writing below `0x30000000`, where there is no physical memory. That access triggers a fault. The CPU tries to report the fault by executing a fault handler, but entering

a handler inherently requires writing to the stack, which fails again. With no way to report the fault, the CPU gives up and enters a state called **lockup**.

Test 2: Unaligned Stack Pointer (0x20000001) If you force SP to an odd address inside RAM, `make run` prints nothing. To see why, start `make debug`, stop right after your `mov sp, r0`, and check the register: `info registers sp` returns `0x20000000`.

The CPU silently drops the two lowest bits of SP because the stack pointer must always be a multiple of 4. The stack now inadvertently starts at the very bottom of RAM and grows *below* it. At `break main`, SP is already under `0x20000000` (e.g., `0x1ffffff0`), in an address range where QEMU simply ignores writes. There is no explicit crash, just no output at all because the function calls fail silently. (Remember to restore `ldr r0, =_estack` before continuing).

1.5.3 Step 4: Zero the .bss Section

The C standard strictly promises that global variables without an explicit initializer (e.g., `int counter;`) will start at `0`[cite: 46]. They physically reside in the `.bss` section, mapped between `_sbss` and `_ebss` in RAM[cite: 46]. Because nothing in flash holds their zeroed value, the boot code must explicitly write these zeros[cite: 46].

This is the exact same looping logic as the data copy, but without a source pointer reading from flash[cite: 46]:

C Equivalent	ARM Assembly

`uint32_t *dst = &_sbss;`	`ldr r1, =_sbss`
	`ldr r2, =_ebss`
	`movs r3, #0`

`while (dst < &_ebss)`	`zero_bss: cmp r1, r2` `bhs zero_bss_done`
`*dst++ = 0;`	`str r3, [r1], #4` `b zero_bss`
	`zero_bss_done:`

This loop is labeled `zero_bss` in `boot/startup.s`[cite: 46]. In the current base skeleton, `.bss` happens to be empty (so `_sbss` equals `_ebss`), meaning the loop exits immediately[cite: 46]. However, the first uninitialized global you add (like `int total;`) will absolutely rely on it[cite: 46].

Warning: Remember that RAM is *not* zeroed at power-on on real hardware[cite: 46]. While the QEMU emulator zeroing RAM for you is a convenience, you must not architecturally count on it[cite: 46].

With steps 1 through 4 completed, `Reset_Handler` replicates everything that the previous C startup code did[cite: 46]. The remaining steps go further: they configure the CPU so that errors are caught, escalated, and reported[cite: 46].

1.6 Exceptions and Faults: One Family

You have already encountered the concept of an **exception**: any hardware or software event that forces the CPU to stop executing the current code and run a specific handler function[cite: 46]. The CPU fetches the address of this handler from the vector table and executes it in Handler mode[cite: 46].

The Cortex-M3 physically defines the following family of exceptions[cite: 46]:

#	**Exception**	**Raised by**
0	*(not an exception)*	This slot holds the initial SP value
1	Reset	Power-on, reset button
2	NMI	Non-maskable interrupt: cannot be ignored
3	**HardFault**	Any fault with nowhere else to go
4	**MemManage**	Breaking memory protection rules
5	**BusFault**	Access to an address where nothing answers, or that refuses access
6	**UsageFault**	Bad use of an instruction: undefined opcode, Thumb bit 0, divide by 0
7–10	*reserved*	Unused: the slot must hold `0`
11	SVCall	The `svc` instruction
12	DebugMonitor	Debug events (unused here)
13	*reserved*	Unused: the slot must hold `0`
14	PendSV	A request raised by software
15	SysTick	The CPU's built-in timer
16+	IRQs	External device interrupts

Slots 7–10 and 13 are strictly reserved[cite: 46]. The four specific faults (3 through 6) are the CPU's direct way of signaling that "the program did something wrong"[cite: 46].

Crucially, at power-on reset, only the `HardFault` exception is actively listening[cite: 46]. The three more specific faults (`MemManage`, `BusFault`, `UsageFault`) are disabled by default[cite: 46]. When a specific fault is disabled, it *escalates* directly to a `HardFault`[cite: 46]. This escalation mechanism explains the "can't escalate 3 to HardFault" lockup message we saw earlier: the CPU was trying to escalate to exception 3 (`HardFault`), but because the handler could not safely push to the stack, it failed[cite: 46].

1.6.1 Completing the Vector Table with Weak Symbols

The base table only has 2 words defined, but the Cortex-M3 expects one word for each exception from 0 to 15 (16 words in total)[cite: 46]. Every slot strictly needs *some* valid address, even for exceptions you have no custom handler for yet, because if that hardware exception triggers, the CPU will blindly jump to whatever address the slot contains[cite: 46]. (The reserved slots must contain `0`)[cite: 46].

Therefore, `isr_vector` in `boot/startup.s` gets 14 additional `.word` entries[cite: 46]. Instead of writing a dozen empty functions for every single exception, the standard engineering practice is to route them all to a generic default loop using weak symbols and aliases in assembly[cite: 47]:

```
.word HardFault_Handler                 @ slot 3, and so on up to
.weak       HardFault_Handler           @ may be replaced from C
```

```
    .thumb_set HardFault_Handler, Default_Handler
Default_Handler:
    b Default_Handler                                   @ for (;;) {}
```

Which translates to the following compiler directive in C[cite: 47]:

```
void HardFault_Handler(void) __attribute__((weak, alias("Default_Handler"
```

How it works:

- `.word HardFault_Handler` places the address of the label into the exact table slot[cite: 47].
- `.thumb_set` dictates to the assembler: "Make `HardFault_Handler` another name for `Default_Handler` (sharing the same address, marked as Thumb code)"[cite: 47].
- `.weak` makes that name a **weak symbol**: a default implementation that easily yields[cite: 47]. If any other file (like `src/main.c`) explicitly defines a normal, non-weak function named `void HardFault_Handler(void)`, the linker silently drops the weak assembly default and routes the hardware exception directly to the developer's custom C function[cite: 47].

`Default_Handler` itself is simply an infinite loop[cite: 47]. An unexpected exception is a critical system bug, and freezing the CPU at a known, predictable place is much easier to diagnose in a debugger than allowing it to run off into random memory addresses[cite: 47].

The Effect on Memory Layout

Completing the vector table immediately alters your memory layout[cite: 47]. The table grows physically from 2 words to 16 words (from 8 bytes to 64 bytes)[cite: 47]. Because the executable code still flows right after it in FLASH, the start of `Reset_Handler` moves down from address `0x8` to `0x40`[cite: 47].

If you attach GDB at reset now, `info registers sp pc` will show `pc 0x40 <Reset_Handler>`, and examining address 0 (`x/2wx 0`) will print `0x0 <isr_vector>: 0x20400000 0x00000041` (proving the Thumb bit is added to the `0x40` address)[cite: 47].

If you examine the `nm` output of the final binary, you will clearly see the effect of the weak symbols: `NMI_Handler`, `PendSV_Handler`, `SysTick_Handler`, and `DebugMon_Handler` are explicitly marked `W` (weak) and all share the exact same memory address as `Default_Handler`[cite: 47]. Meanwhile, any handlers written and compiled from `src/main.c` are firmly marked `T` (Text) at their own unique addresses[cite: 47].

1.6.2 CPU Settings Live at Memory Addresses

The remaining configuration steps change settings inside the CPU core itself. You do not need special assembly instructions for most of them: the Cortex-M architecture makes these settings physically available as 32-bit registers at fixed memory addresses in the system area of the memory map (specifically, from `0xE000E000` to `0xE000EFFF`).

The specific group of registers that controls exceptions is called the **System Control Block (SCB)**. You can change them using ordinary load and store instructions.

C Equivalent	ARM Assembly

`#define SCB_SHCSR (*(volatile uint32_t *)0xE000ED24)`
`SCB_SHCSR |= (1u « 16);`
	`ldr r0, =0xE000ED24 @ addres`
	`ldr r1, [r0] @ read`
	`orr r1, r1, #0x10000 @ modif`
	`str r1, [r0] @ write`

The Read-Modify-Write Pattern and `volatile`

This architectural pattern is known as **read-modify-write**: you read the current value of the register, modify only the specific bits you care about using a bitwise OR, and write the result back. This ensures that all other configuration bits in the register keep their original values.

In the C version, the `volatile` keyword is mandatory. It explicitly tells the compiler that every memory access must truly happen exactly in the order written. Without it, the compiler's optimizer might skip a read (assuming "the value cannot have changed") or remove a write (assuming "nobody reads it"). For hardware registers, both assumptions are completely false.

Debugging tip: You can read these registers from gdb just like any other memory address. For example, `x/wx 0xE000ED00` shows `0x410fc231`, which is the CPUID register identifying the chip as an ARM (`0x41`) Cortex-M3 (`0xc23`).

1.6.3 Step 5: VTOR, Where the Vector Table Is

When an exception happens, the CPU does not technically read slot N strictly at address `4 * N`. It reads it at `VTOR + 4 * N`, where **VTOR** (the Vector Table Offset Register, located at `0xE000ED08`) holds the base address of the table.

Its default value at reset is `0x00000000`, which is exactly why our initial table must reside at address 0. However, the boot code explicitly writes the address of `isr_vector` into it anyway:

C Equivalent	ARM Assembly

`*(volatile uint32_t *)0xE000ED08 =`
`(uint32_t)isr_vector;`
	`ldr r0, =0xE000ED08`
	`ldr r1, =isr_vector`
	`str r1, [r0]`

(Note: The table address must be suitably aligned, at least to 128 bytes, because the low bits of VTOR are not used by the hardware).

Why perform this write if our table is already at 0? Because in real production systems, the table is often located elsewhere. A **bootloader** sitting at address 0 may start an application stored higher in the flash memory, whose own vector table is at a different address. The application must then point VTOR to its own table. Alternatively, an OS may copy its table into RAM to replace handlers dynamically

while running. Explicitly writing VTOR ensures the boot code remains correct regardless of where the final image is placed.

Where you write it: In `Reset_Handler`, right after the label `zero_bss_done:` (the end of step 4). Because you are setting the entire register to a new absolute address, no read-modify-write is needed. In gdb at `main`, `x/wx 0xE000ED08` will show `0x00000000`: the same as the reset value, confirming the table is at address 0 (verified by `p/x &isr_vector`).

1.6.4 Step 6: SHCSR, Separate Fault Handlers

The **SHCSR** (System Handler Control and State Register) at address `0xE000ED24` enables the three specific hardware faults:

Bit	Name	When set
16	MEMFAULTENA	MemManage faults run `MemManage_Handler`
17	BUSFAULTENA	Bus faults run `BusFault_Handler`
18	USGFAULTENA	Usage faults run `UsageFault_Handler`

At reset, these bits are heavily restricted (set to 0), meaning all faults instantly escalate to a `HardFault`. This gives you a single handler for every kind of error, making it impossible to tell *what* specifically went wrong inside it.

The boot code must set bits 16–18 simultaneously with a bitwise OR (`|= 0x00070000`), ensuring each fault routes cleanly to its own dedicated handler. Because the other bits of SHCSR report critical system status (like which handlers are active or pending), you **must** use the read-modify-write pattern and never simply overwrite the whole register.

Where you write it: Right after step 5. You implement the read-modify-write pattern: `ldr r0, =0xE000ED24`, then `ldr r1, [r0]` (read), `orr r1, r1, #0x00070000` (modify), and finally `str r1, [r0]` (write). In gdb at `main`, checking `x/wx 0xE000ED24` will correctly display `0x00070000`.

1.6.5 Step 7: CCR, What Counts as a Fault

CCR, the Configuration and Control Register physically located at `0xE000ED14`, dictates whether certain suspicious CPU operations are classified as faults at all.

Bit	Name	Meaning	Here
3	UNALIGN_TRP	Trap unaligned word/halfword accesses	left off
4	DIV_0_TRP	Trap integer divide by 0 (as a UsageFault)	**set**
9	STKALIGN	8-byte-align the stack on exception entry	already 1

Without the `DIV_0_TRP` bit set, an integer division by zero on the Cortex-M3 does not inherently trigger a fault: the hardware divide instruction silently returns **0**. The program would continue running with a mathematically incorrect value, and nobody would notice. Setting bit 4 transforms that silent, catastrophic bug into an immediate `UsageFault`.

Unaligned memory accesses, on the other hand, are strictly legal on the Cortex-M3 (though slightly slower). While trapping them can help identify sloppy pointer

casts, it can also inadvertently break correct, optimized code produced by the C compiler or standard libraries. Therefore, it is intentionally left off. Bit 9 (`STKALIGN`) is already set to 1 at hardware reset, and the read-modify-write pattern safely preserves it.

Where you write it: Right after step 6. The implementation requires the same four read-modify-write instructions: the address becomes `0xE000ED14` and the bitwise OR utilizes `#(1 « 4)`. The C equivalent is `*(volatile uint32_t *)0xE000ED14 |= (1u « 4);`. Here, read-modify-write is absolutely critical because bit 9 (`STKALIGN`) is already 1 and must remain 1. In gdb at `main`, checking `x/wx 0xE000ED14` will successfully show `0x00000210`.

1.6.6 Step 8: Exception Priorities

When two exceptions compete simultaneously, the more urgent one executes first. An urgent exception can even pre-empt and interrupt a handler that is already currently running for a less urgent one.

On the Cortex-M architecture, **a lower number means a more urgent priority**. Reset (-3), NMI (-2), and HardFault (-1) possess fixed, immutable priorities that are mathematically more urgent than anything a developer can configure. (This finally explains the lockup message encountered earlier: "current priority -1" means the CPU was already executing at the HardFault level, meaning there was physically nowhere higher left to escalate).

Every other exception has a configurable priority, stored as a single byte in the **SHPR** (System Handler Priority Registers):

Register	Byte 0	Byte 1	Byte 2	Byte 3
SHPR1 (`0xE000ED18`)	MemManage	BusFault	UsageFault	—
SHPR2 (`0xE000ED1C`)	—	—	—	SVCall
SHPR3 (`0xE000ED20`)	—	—	PendSV	SysTick

Because each priority is strictly a byte of its own, the boot code writes it directly using a byte store instruction (`strb`). This entirely bypasses the need for the read-modify-write pattern:

C Equivalent	ARM Assembly

`uint8_t *prio = (uint8_t *)0xE000ED18;`
`prio[1] = 0x00; /* BusFault: urgent */`
	`ldr r0, =0xE000ED18`
	`movs r1, #0x00`
	`strb r1, [r0, #1]`

The architectural choice made in `Reset_Handler` is structured as follows: faults are the most urgent (`0x00`), SVCall sits in the middle (`0x80`), and PendSV alongside SysTick are the least urgent (`0xE0`). Real physical chips typically implement only the **upper** 3 or 4 bits of each priority byte (the lower bits always read as 0), so you must purposefully choose hexadecimal values that differ in their top bits, such as these.

Where you write it: Right after step 7. Think of SHPR1, SHPR2, and SHPR3 collectively as a contiguous array of 12 bytes starting at address `0xE000ED18`.

- `prio[0]`, `prio[1]`, and `prio[2]` map to MemManage, BusFault, and UsageFault (set to `0x00`).

- `prio[7]` maps to SVCall (SHPR2 byte 3, which is $4 + 3$; set to `0x80`).
- `prio[10]` and `prio[11]` map to PendSV and SysTick (SHPR3 bytes 2 and 3; set to `0xE0`).

In assembly, you load the base address `0xE000ED18` into `r0` exactly once. Then, for each distinct value, you execute a `movs r1, #value` followed by one `strb r1, [r0, #index]` instruction per byte. In gdb at `main`, checking `x/3wx 0xE000ED18` will explicitly show `0x00000000 0x80000000 0xe0e00000`.

1.6.7 Step 9 and After: PRIMASK and Handing Over to C

The `cpsid i` and `cpsie i` instruction pair operates directly on **PRIMASK**, a special 1-bit register physically located inside the CPU core (it is not mapped in standard memory). When PRIMASK is set to 1, the CPU aggressively ignores every exception that has a configurable priority; only NMI and HardFault are allowed to pass through.

Instruction	PRIMASK State	C (vendor headers)
`cpsid i`	1: masked	`__disable_irq()`
`cpsie i`	0: accepted	`__enable_irq()`

PRIMASK defaults to `0` at reset. The boot code deliberately sets it at the very beginning (Step 1) and meticulously clears it just before calling `bl main`, ensuring that the vector table, handlers, and exception priorities are entirely configured before any interrupts are permitted.

Interrupts that arrive while the system is masked are not discarded or lost: the hardware registers them as **pending**, forcing them to wait, and immediately executes them as soon as PRIMASK is cleared. You can observe the PRIMASK state in gdb by typing `info registers primask`.

After `cpsie i` comes the `bl main` instruction, seamlessly handing execution over to your C code. Following this, the assembly file implements a `hang` label with a `b hang` instruction. This is the architectural equivalent of an infinite `for (;;) {}` loop: on a bare-metal machine, if `main` ever inadvertently returns, there is physically nowhere to return to, so the CPU must spin safely forever.

Where you write it: `cpsid i` is already present as Step 1. Step 9 is a single `cpsie i` instruction, inserted right after Step 8 and just before `bl main`. In gdb, inspecting `info registers primask` will correctly show `0` at reset, `1` after a single `stepi` (past `cpsid i`), and `0` again once execution reaches `main`.

1.6.8 Checking the Configuration from GDB

Once steps 4 through 9 are written, you can definitively prove that all hardware bits are correctly configured by the time `main` starts. With `make debug` running in your first terminal, execute the following in GDB:

```
(gdb) target remote :1234
(gdb) info registers primask     # 0 at reset; after stepping past cpsid i
(gdb) break main
(gdb) continue
```

```
(gdb) x/wx 0xE000ED08      # VTOR
(gdb) x/wx 0xE000ED24      # SHCSR
(gdb) x/wx 0xE000ED14      # CCR
(gdb) x/3wx 0xE000ED18     # SHPR1..3
```

The expected memory values are precisely as follows:

- **VTOR:** `0x00000000` (unchanged, meaning the table is actively at address 0).
- **SHCSR:** `0x00070000`, which exactly corresponds to bits 16, 17, and 18 being active.
- **CCR:** `0x00000210`, which is bit 4 (`0x10`, the division-by-zero trap you added) plus bit 9 (`0x200`, the stack alignment bit already set at reset).
- **SHPR:** `0x00000000 0x80000000 0xe0e00000`.

Reading the SHPR byte by byte: Because the Cortex-M architecture is little-endian, byte 0 is the rightmost pair of hex digits.

- **SHPR1** is all zeros (the three specific faults are set to the highest configurable priority).
- **SHPR2** has `0x80` strictly in byte 3 (corresponding to SVCall).
- **SHPR3** has `0xe0` strictly in bytes 2 and 3 (corresponding to PendSV and SysTick, the lowest priorities).

1.6.9 Catching a Divide by Zero

The configuration performed in the `Reset_Handler` is only useful if it successfully catches real errors in the code. To prove this, we can write custom fault handlers in C and intentionally trigger a fault.

In `src/main.c` (right after `data_message` and before `main`), you can write four functions utilizing the exact names defined in the vector table: `HardFault_Handler`, `MemManage_Handler`, `BusFault_Handler`, and `UsageFault_Handler`. Each function prints its own name using the same `semihost_write0` function that `main` uses, and then enters an infinite loop:

```
void UsageFault_Handler(void)
{
    semihost_write0("UsageFault_Handler: a usage fault happened\n");
    for (;;) { }
}
```

Because these are normal (strong) C definitions, the linker uses them to silently replace the weak assembly defaults established in `boot/startup.s`. They loop forever because a fault represents a critical bug; there is no sensible way to continue execution safely.

Triggering the Fault

To test the handlers, we can force `main` to divide by zero. Crucially, the divisor must be a `volatile` variable holding `0`: without `volatile`, the compiler's optimizer will detect the constant division, compute or remove it entirely at compile time, and no physical divide instruction will ever be executed by the CPU.

Using a preprocessor switch allows us to toggle the demo safely:

```
#define DEMO_DIVIDE_BY_ZERO 1

/* Inside main() */
#if DEMO_DIVIDE_BY_ZERO
    volatile uint32_t numerator = 42;
    volatile uint32_t zero = 0;
    volatile uint32_t result = numerator / zero;
    (void)result; /* Suppresses the 'unused variable' compiler warning */
#endif
```

Running `make run` will successfully halt the program and print:

```
UsageFault_Handler: a usage fault happened
```

Validating the Configuration Steps

The two hardware configuration steps we added each play a distinct role, which can be verified by disabling them one at a time:

- **Without Step 6 (SHCSR bits):** The specific usage fault is disabled. The hardware escalates the error, and you see `HardFault_Handler: a hard fault happened`. The critical information regarding *what* specific kind of fault occurred is permanently lost.
- **Without Step 7 (DIV_0_TRP):** You see no fault message at all. The division silently returns `0`, and `main` continues executing its usual lines as if nothing happened. This is precisely the silent, catastrophic bug the trap was configured to catch.

1.6.10 What Real Chips Add (`SystemInit`)

The QEMU emulator's CPU is ready to run at full speed the moment it leaves reset. However, on a real physical microcontroller, the startup code usually must perform significantly more initialization before `main` is called. This is typically done in a vendor-supplied function called `SystemInit()`, which is called directly from `Reset_Handler`.

Even though our emulator does not strictly require them, you should understand what a standard `SystemInit()` typically handles:

- **Clock and PLL:** The chip naturally starts on a slow internal oscillator for stability. A Phase-Locked Loop (PLL) circuit must be configured to multiply that frequency up to the chip's maximum operational speed.

- **Flash Wait States:** Physical Flash memory is fundamentally slower than a high-speed CPU. Before ramping up the clock frequency, the code must instruct the flash controller to insert extra "wait cycles" into each memory access; otherwise, the CPU would fetch invalid garbage data.
- **Watchdog Timer:** A watchdog is a hardware timer that automatically resets the chip unless the software periodically "feeds" (resets) it. This acts as a physical safeguard against a program that hangs. On some chips, the watchdog is active immediately upon reset, so the boot code must either explicitly turn it off or begin feeding it.
- **FPU (Floating-Point Unit):** On architectures like the Cortex-M4F, the FPU is disabled at reset to save power. It must be explicitly enabled via the **CPACR** register (`0xE000ED88`); otherwise, executing the very first `float` instruction will trigger a `UsageFault`.
- **C++ Static Constructors:** If writing an OS or application in C++, the startup code must explicitly iterate through the `.init_array` section and call all static object constructors before jumping to `main`.

1.7 Life Without an Operating System

Your program now boots entirely on code you explicitly wrote. This unit examines what is *not* there and the architectural consequences of those missing layers. It culminates with the program's final structural form: `main` intentionally relinquishing its privileges and then requesting privileged work back through a formal system call.

1.7.1 Hosted vs. Freestanding C Environments

The C standard strictly defines two distinct execution environments for any C program.

	Hosted	**Freestanding**
Example	Your laptop program	Our Cortex-M3 image
Under your code	A C library (**libc**) and an OS	Nothing
`printf`, `malloc`, `fopen`, `exit`	Available	Not available
Who starts `main()`	libc startup code, called by the OS	Whoever you decide

On a standard laptop, writing `printf("hi\n")` "just works" because two massive, complex layers sit directly beneath your program doing the heavy lifting: the C library and the Operating System. Our embedded image operates in a **freestanding** environment. Below your bare-metal code, there is absolutely nothing.

What is Available in Freestanding C?

A freestanding environment still provides a subset of standard headers. However, these are strictly headers that require **no code execution at run time**. They provide only data types, architectural constants, and macros that the compiler resolves entirely at build time.

Header	Provides
`<stdint.h>`	`uint32_t`, `int8_t`, ...
`<stddef.h>`	`size_t`, `NULL`, `offsetof`
`<stdbool.h>`	`bool`, `true`, `false`
`<stdarg.h>`	`va_list`, for variadic functions
`<limits.h>`, `<float.h>`	`INT_MAX`, `FLT_EPSILON`, ...

This is precisely why `src/main.c` only includes `<stdint.h>`. There is no `<stdio.h>`, no `<stdlib.h>`, and no `<string.h>`: this means no printing, no dynamic allocation (`malloc`), no file I/O, no `exit`, and not even `memcpy`. Anything else required by the system, you must write yourself. (This lack of standard library support is the exact reason the assembly boot code had to manually copy the `.data` section with its own explicit loop rather than just calling a convenient `memcpy`).

1.7.2 What is Really Under `printf`

To understand why standard functions fail without an OS, it helps to dissect what `printf` actually does under the hood on a hosted system (like a laptop):

```
printf("x=%d\n", x)            your program
    |  format the text into a buffer          |   v
| libc
write(1, buf, len)                             |
    |  system call: the CPU enters the kernel   |
    v                                           | OS kernel
terminal / file / serial device driver         |
    |
    v
hardware (screen, UART, disk)
```

`printf` itself only performs **formatting**: it takes a format string like `"x=%d\n"` and the value of `x`, and parses them into a raw sequence of ASCII characters stored in a memory buffer.

However, getting those physical characters out of the machine to a screen is not `printf`'s job. To do that, it calls `write()`, which triggers a **system call**. A system call forces the CPU to enter the Operating System kernel. The kernel's specific device driver then takes over, physically sending the bytes to the terminal, a file, or a serial hardware device.

`printf` is not the only C library feature that relies on hidden, lower-level OS hooks:

libc feature	What it needs underneath
`printf`, `puts`	`write()`: a way to send bytes to a physical device
`malloc`, `free`	a heap memory region, dynamically grown via `sbrk()`
`exit`	an OS process manager to return control to
`main()` being called	startup code (**crt0**): sets up the stack, `.data`, `.bss`, then calls `main`

C libraries designed for embedded systems (such as **newlib**) implement the high-level formatting logic and data structures, but they intentionally leave these bottom-

level hardware functions as empty **hooks** (stubs) that the developer must manually implement for their specific physical board.

1.7.3 Why This Course Uses No libc At All

There are three fundamental reasons we deliberately avoid linking any C library at all:

1. **No Underlying Kernel Hooks:** Without an OS beneath us, nothing implements those required hooks. There is no kernel, no `write()` function, and no heap manager; if we linked libc, we would have to write these complex low-level stubs first anyway just to satisfy the linker.

2. **Redundant Startup Logic:** The job of `crt0` (the standard libc startup code that prepares the C environment) is already perfectly handled by the custom `Reset_Handler` we built in `boot/startup.s` (setting up the stack, copying `.data`, clearing `.bss`, and branching to `main`).

3. **Absolute Visibility:** The primary educational goal is to **see every single layer**. Every byte that executes on the board is code you can actively read in the repository. The final compiled image contains exclusively `boot/startup.s`, `src/main.c`, and occasionally a few necessary compiler helper routines. Absolutely nothing is hidden from the developer.

1.7.4 Bypassing `libc`: The `printf` Problem

If there is no `libc` and no physical UART driver, how did our bare-metal program successfully print lines to the laptop's terminal[cite: 37]?

Why not just use standard `printf`? Our hand-rolled `Reset_Handler` does not perform the massive C-runtime initialization that `newlib`'s semihosting-backed standard I/O (`rdimon`) expects. Simply routing through `printf` would silently fail and produce no output. Instead, we use **direct semihosting** to sidestep the C library entirely, which our freestanding build does not link against anyway.

1.7.5 Semihosting: How the Program Still Prints

Semihosting is a hardware debug convention where the embedded program intentionally traps to the debugger (here, QEMU) to ask it to perform host I/O operations on the target's behalf[cite: 37].

Instead of driving real physical hardware, the execution follows this precise sequence[cite: 37, 38]:

1. The program places an operation code into register `r0`. Here, it uses `0x04`, known as `SYS_WRITE0`, meaning "print this string terminated by a zero byte"[cite: 37].

2. It places the memory address of the string into register `r1`[cite: 37].

3. It executes `bkpt 0xab`: a software breakpoint instruction tagged with a special identifying number[cite: 38].

4. QEMU intercepts this breakpoint, halts the CPU, reads `r0` and `r1`, fetches the string directly from the target's RAM, and prints it on your laptop's terminal[cite: 38].

5. Once finished, the host commands the program to resume execution seamlessly[cite: 38].

1.7.6 Implementing Semihosting in C (`main.c`)

To achieve this without an operating system, the C implementation in `src/main.c` uses Extended Inline Assembly to force the CPU to fulfill this hardware contract:

```
#include <stdint.h>

static void semihost_write0(const char *message)
{
    /* 1. Operation Code into r0 */
    register uint32_t r0 __asm__("r0") = 0x04;

    /* 2. String Address into r1 */
    register const char *r1 __asm__("r1") = message;

    /* 3. Execute the hardware trap */
    __asm__ volatile ("bkpt 0xab" : : "r"(r0), "r"(r1) : "memory");
}
```

This small function introduces advanced compiler features:

- **Register Binding:** The `register ... __asm__("r0")` syntax is a GCC extension that forces the compiler to place the value into that exact physical CPU register. Standard C does not guarantee which register a variable will use, but the semihosting convention rigidly demands `r0` and `r1`[cite: 38].

- **The Instruction:** `__asm__ volatile` allows the insertion of the raw `bkpt 0xab` assembly instruction, which has no C equivalent[cite: 38].

- **Memory Barrier:** The `"memory"` clobber at the end is a critical safeguard. It warns the compiler that this instruction may arbitrarily read memory (the string). This explicitly prevents the C optimizer from reordering earlier variable writes past this point, guaranteeing the string is fully assembled in RAM before QEMU halts the CPU to read it[cite: 38].

1.7.7 Semihosting is a Trap

Look closely at what `bkpt 0xab` really does[cite: 38]. The program stops doing its own work; control passes to someone more powerful (QEMU, outside the emulated CPU); that someone performs the job and hands control back[cite: 38, 39].

This is the exact same architectural shape as a system call on Linux: your program cannot drive the terminal itself, so it asks the kernel to do it, and the kernel returns control when the job is done[cite: 39].

1.7.8 Privileged and Unprivileged Code on Cortex-M

The Cortex-M3 implements the kernel/user mode concept directly in hardware, combined with the two CPU modes[cite: 79]:

CPU mode	When	Privilege
Handler mode	running an exception handler	always **privileged**
Thread mode	normal code (`main`, ...)	privileged **or** unprivileged

After reset, the CPU defaults to Thread mode and is **privileged**[cite: 80]. That is exactly how `Reset_Handler` was able to freely write to VTOR, SHCSR, and CCR[cite: 80]. **Unprivileged** Thread mode is the equivalent of user mode: the restricted mode for code that a kernel would not trust[cite: 80].

The CONTROL Register

Which privilege level Thread mode uses is selected by the **CONTROL** register, a special register inside the CPU (like PRIMASK, it is not mapped in standard memory)[cite: 81]:

Bit	Name	Meaning
0	**nPRIV**	0 = privileged, 1 = unprivileged (Thread mode only)
1	SPSEL	selects which stack pointer Thread mode uses (not used here)

Because special registers are not reachable with standard `ldr` and `str` instructions, two dedicated instructions are used to read and write them[cite: 81]:

Instruction	C equivalent
`mrs r0, control`	`r0 = CONTROL;` (read a special register)
`msr control, r0`	`CONTROL = r0;` (write a special register)

1.7.9 Dropping Privilege: A One-Way Street

In `src/main.c`, after the first two lines are printed, the program intentionally drops its own privilege using a small block of inline assembly[cite: 81]:

ARM Assembly	C Equivalent

`mrs r0, control`
`orr r0, r0, #1`
`msr control, r0`
`isb`
	`CONTROL = CONTROL | 1;`

This is a read-modify-write pattern, this time executed on a special register[cite: 81]. The last instruction, `isb` (Instruction Synchronization Barrier), is critical because modern CPUs fetch instructions ahead of time[cite: 81]. Some of the following instructions may have already been fetched under the old, privileged rules[cite: 81]. `isb` flushes the pipeline, throws those pre-fetched instructions away, and forces the CPU to fetch them again under the strictly enforced unprivileged rules[cite: 81].

The Privilege Trap: This transition is a **one-way street**[cite: 81]. Writing to the CONTROL register is itself a privileged operation: if unprivileged code attempts to execute `msr control, r0` to clear nPRIV, the hardware simply **ignores** the write[cite: 81]. Code cannot maliciously promote itself back to privileged execution[cite: 81, 82]. There are only two ways back: a full hardware **reset**, or **an exception** (because exception handlers automatically run in Handler mode, which is unconditionally privileged)[cite: 82].

1.7.10 What Unprivileged Code Cannot Do (The Sandbox Constraints)

Once restricted, the CPU actively blocks critical operations to create a secure sandbox[cite: 83]:

Attempt	Result
Read or write the **System Control Space** (`0xE000E000`...): SysTick, NVIC, SCB	**BusFault**
`msr control, r0` (raise privilege back)	Ignored
`cpsid i` / `cpsie i` (mask or unmask interrupts)	Ignored
Semihosting `bkpt 0xab`	QEMU refuses it, so it faults

(Note: The NVIC is the hardware block that manages device interrupts; it lives in the same restricted system area as the SCB)[cite: 83].

Ordinary RAM, flash memory, and standard code execution still work normally[cite: 83]. The restricted code simply cannot **reconfigure the machine**, and it cannot silence it by masking interrupts[cite: 83].

Try It: The HardFault Escalation

The semihosting restriction has a direct, observable consequence[cite: 83]. Right after the privilege-drop block in `main`, add a call to `semihost_write0("main: unprivileged hello\n");` and run `make run`[cite: 83]:

```
main: privileged hello
main: this text lives in .data
HardFault_Handler: a hard fault happened
```

Your line never prints[cite: 84]. QEMU strictly refuses semihosting requests from unprivileged code[cite: 84]. The `bkpt` instruction is treated as a security violation and triggers a fault[cite: 84]. Since no dedicated fault handler covers this specific trap in the current setup, it escalates to **HardFault**[cite: 84].

Notice that `HardFault_Handler` *can* print via semihosting, even though the code that caused the fault was unprivileged[cite: 84]. This is because handlers automatically run in Handler mode, regaining full privileges[cite: 84]. (Stop QEMU with `Ctrl-C` and remove the offending line).

Debugging the CONTROL Register

You can watch the `nPRIV` bit flip in the debugger[cite: 80]. With `make debug` in terminal 1, run the following in GDB:

```
(gdb) target remote :1234
(gdb) break main
(gdb) continue
(gdb) p/x $control          # 0x0: privileged
```

Then let the program run past the privilege drop (for example, by setting a breakpoint on `SVC_Handler`—introduced in the next section—and continuing)[cite: 80]. Print `$control` again: it is now `0x1`[cite: 80]. The `nPRIV` bit is set, and it securely stays set[cite: 80].

1.7.11 SVC: The Door That Stays Open

Unprivileged code is strictly blocked from touching the hardware, but it can still **ask** for tasks to be done on its behalf. The `svc #n` ("supervisor call") instruction is deliberately allowed to execute within unprivileged code.

When executed, it raises the **SVCall** exception (Exception 11), whose designated handler resides in slot 11 of the vector table: `SVC_Handler`. As with any exception, the CPU immediately transitions into Handler mode, meaning the handler itself runs fully **privileged**. When the handler function returns, the CPU drops back into Thread mode to execute the instruction right after the `svc`, **still entirely unprivileged**.

```
Thread mode, UNPRIVILEGED                    Handler mode, PRIVILEGED
-------------------------                    ------------------------

main:
  ...
  svc #0  ------ SVCall exception ------> SVC_Handler:
                                            semihost_write0("...")
                                            (can print, can touch
                                             SysTick, NVIC, SCB)
  next instruction <---- exception return ---- return
  ... (still unprivileged)
```

Hardware privilege is **borrowed** only for the strict duration of the handler, and only the specific code *you* compiled into the handler runs with it. The unprivileged code does not get to choose what the handler runs; it can only knock on the door.

The number `#n` inside `svc #n` is encoded directly into the machine instruction. Real OS kernels utilize this number (combined with arguments passed in registers) to determine *which* specific system service is being requested. For our baseline proof of concept, there is only one service, so the program simply executes `svc #0`.

1.7.12 This Is a System Call

Compare this bare-metal sequence directly with what happens on a standard desktop environment:

The architectural mechanism remains identical at every scale: the restricted side cannot safely perform the job itself, so it intentionally traps into code that can, and

	Linux on your laptop	**Our Cortex-M3**
Restricted code	your program, in user mode	`main` after `nPRIV` = 1
Trap instruction	`syscall` (x86-64), `svc` (ARM)	`svc #0`
Privileged side	the kernel	`SVC_Handler`
Service	`write()` puts bytes on the terminal	print via semihosting

that privileged code securely decides what to do. The `write()` system call located at the bottom of the libc `printf` stack on your laptop works exactly this way.

1.7.13 Adding the System Call

The final steps to turn this into working code happen in `src/main.c`.

First, you must write the `SVC_Handler`. It prints a message using `semihost_write0` and then simply returns:

```
void SVC_Handler(void)
{
    semihost_write0("SVC_Handler: privileged code printing on behalf of u
}
```

Unlike the hardware fault handlers you wrote earlier, it does *not* loop forever: an `svc` trap is a deliberate request for service, not a bug. Falling off the end of the C function triggers a clean return from the exception, dropping privileges and jumping back to the instruction after `svc #0`. Its name strongly overrides the weak default defined in `boot/startup.s`, just like the fault handlers did.

(Note: A standard C function can serve flawlessly as an exception handler because the Cortex-M hardware automatically saves and restores the registers a C function might modify upon exception entry and return).

Then, after the privilege drop in `main`, you insert the trap instruction using inline assembly:

```
__asm__ volatile ("svc #0" : : : "memory");
```

Executing `make run` now correctly prints the third line:

```
SVC_Handler: privileged code printing on behalf of unprivileged code
```

Troubleshooting note: If you forget to explicitly write the `SVC_Handler` in C, the `svc` instruction seamlessly lands in the weak `Default_Handler` defined in `boot/startup.s`, and the program hangs silently in its infinite loop (meaning the weak default successfully did its job).

1.7.14 Touching the Hardware Without Privilege

Finally, you must definitively verify that the unprivileged hardware protection is actively enforced.

After the `svc #0` line, while `main` is still unprivileged, attempt to write `0` into the SysTick timer's control register (located at address `0xE000E010` in the restricted System Control Space):

```
volatile uint32_t *systick_ctrl = (volatile uint32_t *) 0xE000E010;
*systick_ctrl = 0;
```

Running `make run` yields:

```
SVC_Handler: privileged code printing on behalf of unprivileged code
BusFault_Handler: a bus fault happened
```

The output proves that the `svc` successfully returned to unprivileged code. Then, the System Control Space hardware actively rejected the unprivileged store instruction, forcing the CPU to raise a **BusFault**. Because `Reset_Handler` enabled bus faults earlier in the SHCSR, the fault routes directly to its own specific handler, and you immediately know exactly what kind of error occurred.

Conclusion: Privilege and CPU Modes

One final observation completes the picture. In gdb, put a breakpoint on `SVC_Handler`, `continue` execution, and print `$control` inside the handler: it is still `0x1`.

The handler is privileged *not* because the `nPRIV` bit magically cleared itself, but because the CPU is currently running in **Handler mode**. Handler mode is architecturally designed to always be privileged, regardless of what the `CONTROL` register says.

1.8 Conclusion: The Final Result

With all the steps in place, the solution's `make run` prints the four lines you saw at the beginning of this chapter:

```
main: privileged hello
main: this text lives in .data
SVC_Handler: privileged code printing on behalf of unprivileged code
BusFault_Handler: a bus fault happened
```

Read them as a definitive summary of the week's architectural work:

- The **first line** shows that `Reset_Handler` successfully reached `main`, operating securely in privileged Thread mode.
- The **second line** shows that the `.data` copy loop works: the linker's LMA/VMA memory split and your assembly boot code perfectly agree.
- The **third line** shows that after intentionally dropping privilege, unprivileged code can still securely get privileged work done through `svc` and `SVC_Handler`: establishing a true system call.
- The **fourth line** shows that unprivileged code attempting to reconfigure the CPU is immediately stopped by a `BusFault`, which is then cleanly reported by its own dedicated handler thanks to the SHCSR configuration.

Note: With `DEMO_DIVIDE_BY_ZERO` in `src/main.c` set to 1, the same image instead prints `UsageFault_Handler: a usage fault happened`, proving the CCR divide-by-zero trap actively works.

Your program now runs completely **freestanding**: there is no libc, no crt0, and no underlying OS (and the `Makefile` flags state this explicitly). It prints through semihosting, trapping into a more powerful host. Crucially, it implements—in just

a few dozen lines of assembly and C—the core idea of every operating system's protection model: **restricted code cannot touch the machine directly, and must ask privileged code through a single, controlled door.**

1.9 Appendix: Where to Find the Code

The project directory (`week-02/`) holds two sibling folders: `base/` and `solution/`. Each has its own `Makefile` and builds completely independently. Unless a path in these notes is explicitly written with a `base/` prefix, files like `boot/startup.s`, `src/main.c`, `scripts/mps2_m3.ld`, and `Makefile` are relative to `week-02/solution/`.

1.9.1 The Base Skeleton

The starting point of the in-class work is `week-02/base/`:

- `base/boot/startup.s` holds only the minimal two-word vector table and a `Reset_Handler` that blindly calls `main`.
- `base/src/main.c` prints the string literal and `data_message`.
- The linker script and `Makefile` are identical to the solution.

1.9.2 The Finished Result

The fully completed bootloader is in `week-02/solution/`:

- `boot/startup.s` contains the full 16-entry vector table, the weak default handlers, and the completely manual `Reset_Handler` (handling the interrupt mask, stack pointer, `copy_data`, `zero_bss`, VTOR, SHCSR, CCR, exception priorities, and finally `bl main`).
- `src/main.c` contains `semihost_write0`, the four strong fault handlers, the `SVC_Handler`, and the `main` logic that drops privilege, calls `svc #0`, and pokes the SysTick timer to test the sandbox.
- `scripts/mps2_m3.ld` and `Makefile` enforce the freestanding memory layout mapped in Units 1 and 3.

Final note: If these lecture notes and the codebase ever disagree, the C and Assembly code is always the absolute reference.